МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский ядерный университет «МИФИ»
(НИЯУ МИФИ)

ИНСТИТУТ ИНТЕЛЛЕКТУАЛЬНЫХ КИБЕРНЕТИЧЕСКИХ СИСТЕМ

КАФЕДРА «КОМПЬЮТЕРНЫЕ СИСТЕМЫ И ТЕХНОЛОГИИ» (№12)

ОТЧЕТ
КУРСОВОЙ ПРОЕКТ (СХЕМОТЕХНИКА ЦИФРОВЫХ УСТРОЙСТВ)

Семестр 5

Тема: «Сортировщик продукции на конвейере»

Студенты Коротаев С. А., Моря К. О. Группа С24-501
ФИО

Руководитель Решетько В. М.
ФИО, должность, звание, степень

Студент _____ Коротаев С. А.
подпись ФИО

Студент _____ Моря К. О.
подпись ФИО

Руководитель _____ Решетько В. М.
подпись ФИО

Оценка _____

Баллы, ECTS _____

«____»_____2026 г.

Москва 2026 г.

ОГЛАВЛЕНИЕ

1. Введение 3
2. Постановка задачи 3
3. Условное графическое обозначение и порты 3
4. Архитектура устройства 4
4.1. Ядро сортировщика 4
4.2. Демонстрационный генератор 4
4.3. Верхний уровень платы 4
5. Алгоритм работы 5
6. Тактовая организация 6
7. Функциональное моделирование 7
7.1. Организация проверки 7
7.2. Временные диаграммы 7
8. Синтез и подготовка к проверке платы 8
9. Заключение 9
10. Приложение 10

1. Введение

Цель проекта - реализовать на ПЛИС сортировщик продукции на конвейере. Два датчика фиксируют положение изделия, а восьмиразрядный код веса используется для принятия решения. Устройство сохраняет разрешение конвейера и подаёт отдельную команду отбраковки.

2. Постановка задачи

Сортировщик начинает измерение при срабатывании любого датчика. Совместная активность датчиков отмечает прохождение центра. При первом спаде веса после центра устройство запоминает предыдущий отсчёт и сравнивает его с границами 95 и 105 включительно. При выходе за границы формируется импульс отбраковки заданной длительности. Следующее изделие можно обрабатывать после освобождения обоих датчиков.

3. Условное графическое обозначение и порты

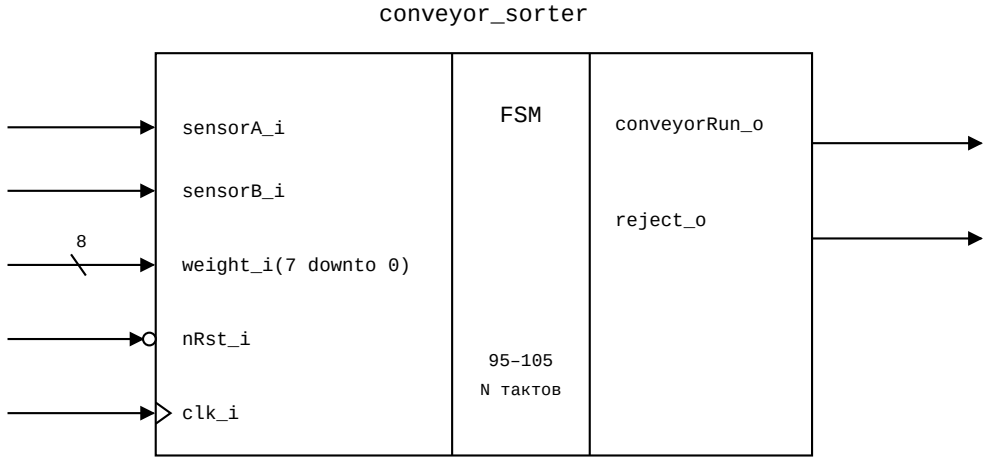


Рис. 1. Условное графическое обозначение ядра

Таблица 1. Порты ядра

Сигнал	Порт	Разрядов	Назначение
clk_i	in	1	Такт, положительный фронт
nRst_i	in	1	Асинхронный сброс, активный 0
sensorA_i	in	1	Датчик A, активная 1
sensorB_i	in	1	Датчик B, активная 1
weight_i	in	8	Беззнаковый код веса 0–255
conveyorRun_o	out	1	Разрешение конвейера
reject_o	out	1	Команда отбраковки

4. Архитектура устройства

4.1. Ядро сортировщика

В `conveyor_sorter.vhd` один тактовый процесс выполняет переходы автомата по `case`, второй обновляет данные и счётчик. Сигнал `captureWeight_s` одновременно разрешает фиксацию веса и переход к его проверке. Сигнал `weightValid_s` определяет переход к отбраковке и разрешает загрузку её счётчика. Оба условия заданы конкурентными присваиваниями. Для беззнакового сравнения весовых кодов применяется `IEEE.STD_LOGIC_UNSIGNED`.

Таблица 2. Регистры ядра

Регистр	Тип	Назначение
`state_r`	`state_type`	Текущее состояние
`previousWeight_r`	8 бит	Последний отсчёт до фиксации веса
`measuredWeight_r`	8 бит	Зафиксированный код веса
`centerFlag_r`	1 бит	Совместная активность датчиков была замечена
`rejectCount_r`	`natural`	Обратный счёт от N − 1 до нуля

4.2. Демонстрационный генератор

`flow_tester.vhd` содержит таблицу `PROFILE` из 16 позиций. Строка задаёт уровни датчиков A и B и два кода веса. Позиции 0, 1 и 9–15 пустые, их датчики и вес равны нулю. После сброса на выходах позиция 0 годного профиля. Обратный счётчик `nextPosition_r` задаёт следующую строку по индексу `15 - nextPosition_r`. При нуле счётчика шага генератор регистрирует выбранную строку, а после выдачи последней позиции переключает профиль.

Таблица 3. Активные позиции профиля

Позиция	A	B	Вес годного	Вес брака	Этап
2	1	0	20	20	Вход
3	1	0	50	30	Рост веса
4	1	1	80	60	Центр обнаружен
5	1	1	100	80	Отсчёт до спада
6	1	1	98	78	Первый спад
7	0	1	70	50	Выход
8	0	1	30	30	Выход

4.3. Верхний уровень платы

`qmtech_demo_top.vhd` соединяет генератор и ядро, подаёт им общий такт и согласует уровни со светодиодами платы. Два регистра синхронизируют снятие кнопочного

сброса. Нажатие кнопки устанавливает сброс асинхронно. D1 и D2 имеют активный низкий уровень, поэтому их входы получают инвертированные `conveyorRun_o` и `reject_o`.

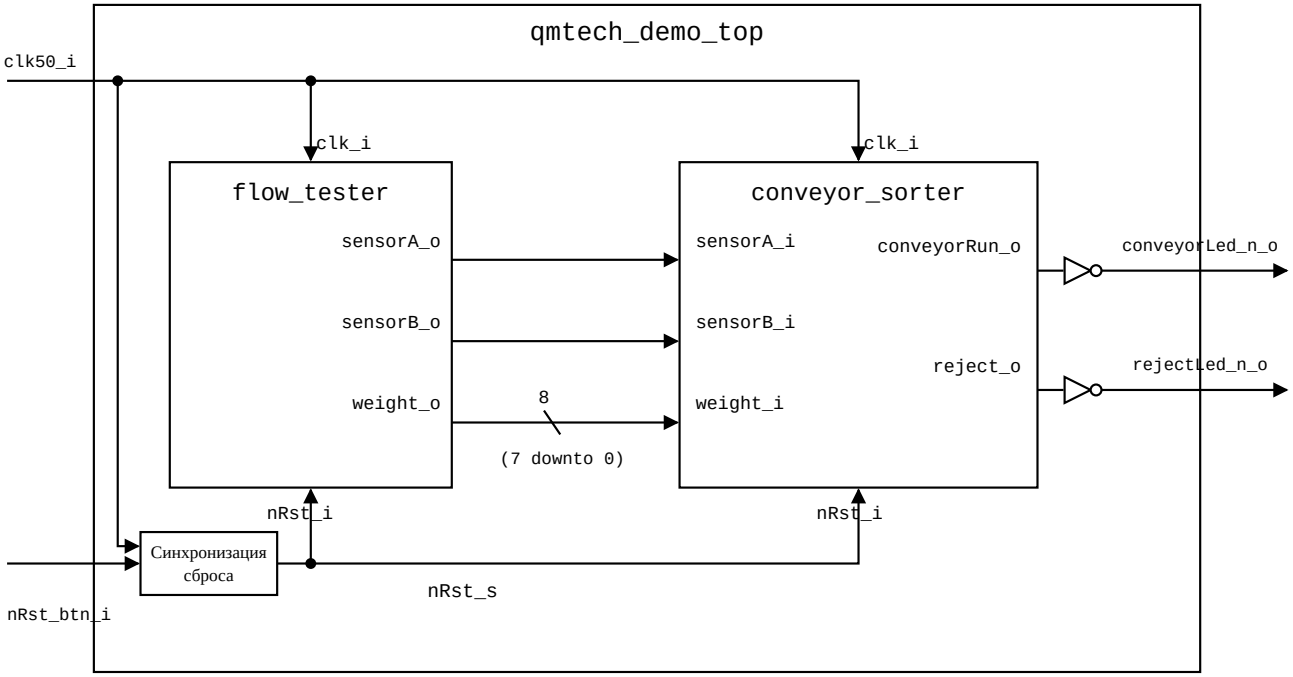


Рис. 2. Структура демонстрационного проекта для платы

В отдельной обёртке `conveyor_sorter_top.vhd` предусмотрены двухступенчатые синхронизаторы внешних датчиков. Фильтр веса обновляет выход при совпадении двух последовательных синхронизированных кодов. При использовании обёртки источник должен обеспечивать согласованность шины и удерживать вес с учётом задержки фильтра. В проекте Quartus верхним уровнем выбран `qmtech_demo_top`.

5. Алгоритм работы

При сбросе автомат переходит в `WAIT_STATE`, регистры данных обнуляются, выход отбраковки выключается. Первый фронт с активным датчиком запускает измерение и записывает текущий вес. Если на этом фронте активны оба датчика, сразу запоминается прохождение центра.

При первом спаде веса после центра фиксируется предыдущий отсчёт. Если датчики освободились после центра при неизменном или возросшем весе, фиксируется последний отсчёт, полученный до освобождения. Если датчики освободились до центра, неполный проход прекращается и автомат готов принять следующее изделие.

На диаграмме A и B обозначают датчики, W - текущий код веса, V - предыдущий код, M - зафиксированный код, F - флаг центра, C - счётчик отбраковки. Условие P выполняется при W < V, если F = 1 или A = B = 1. Условие Q выполняется при A = B = 0 и F = 1.

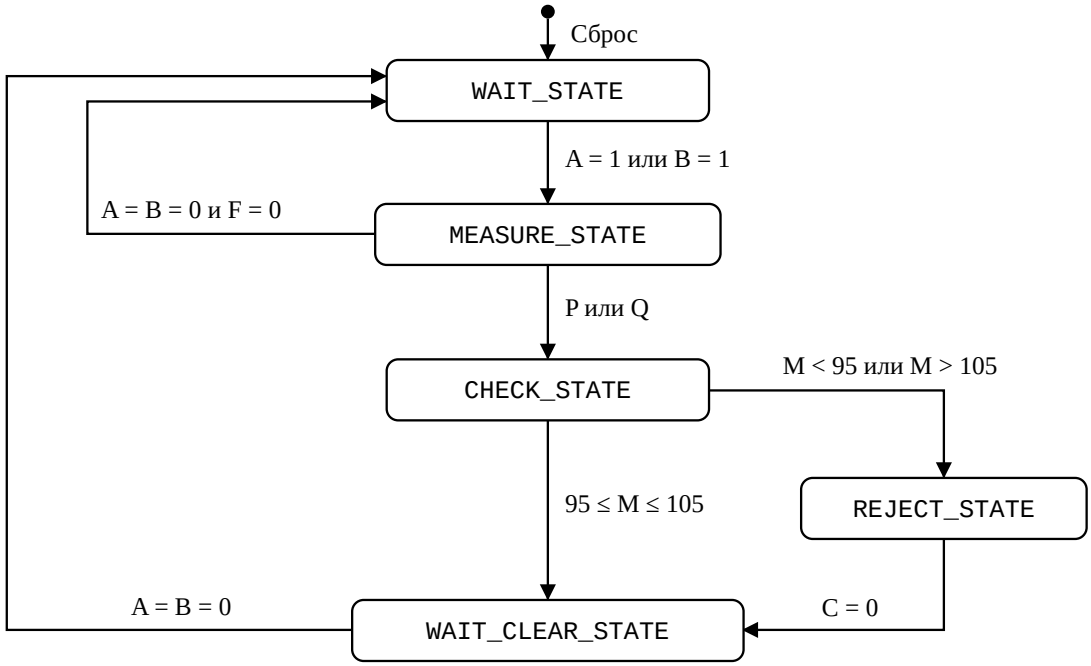


Рис. 3. Переходы автомата при границах 95 и 105

Таблица 4. Действия автомата

Состояние	**Действия**
`WAIT_STATE`	Обновляет V и устанавливает F по текущим датчикам.
`MEASURE_STATE`	Запоминает совместную активность датчиков. При P или Q сохраняет M ← V, иначе обновляет V.
`CHECK_STATE`	Сравнивает M с границами. При браке загружает C ← N − 1.
`REJECT_STATE`	Удерживает `reject_o` = 1. Пока C > 0, уменьшает C на каждом фронте.
`WAIT_CLEAR_STATE`	Ожидает освобождения обоих датчиков.

При отсутствии перехода состояние сохраняется. Асинхронный сброс действует из любого состояния. Состояние проверки занимает один такт, состояние отбраковки - N тактов, включая такт с C = 0. Сигнал `conveyorRun_o` постоянно равен 1.

6. Тактовая организация

Оба модуля демонстрации работают на 50 МГц, период такта 20 нс. Счётчик шага считает от `STEP_CYCLES - 1` до нуля и разрешает обновление выходов генератора по общему такту. На плате шаг равен 5 000 000 тактов, или 100 мс. Один профиль

длится 1,6 с. При чередовании годного изделия и брака период между импульсами отбраковки равен 3,2 с, длительность импульса - 100 мс.

7. Функциональное моделирование

7.1. Организация проверки

`struct_tb.vhd` подаёт на ядро датчики и коды веса и проверяет реакции на каждом существенном фронте. `flow_tester_tb.vhd` соединяет генератор с тем же ядром. Эталонные профили в тестбенче заданы независимо от таблицы генератора. На каждом такте проверяются датчики, вес и точное окно импульса отбраковки. После сброса тест снова ожидает первым годное изделие.

Таблица 5. Сценарии проверки ядра

Воздействие	**Результат**
Вес 95, 100 и 105	Изделие проходит без отбраковки.
Вес 0, 1, 80, 94, 106 и 255	Отбраковка ровно на 1, 2, 3 или 8 тактов согласно параметру.
Оба датчика активны один такт при входе	Центр обнаружен, при освобождении датчиков фиксируется последний отсчёт.
Датчики освободились до центра	Неполный проход пропущен, следующее изделие проверено без сброса.
Датчики удерживаются после проверки	Повторный импульс отсутствует до освобождения зоны.
Сброс во время отбраковки	Импульс прекращается асинхронно.

Все семь запусков GHDL 6.0.0 завершились успешно. В четырёх запусках варьировалась длительность импульса ядра, в трёх совместных запусках шаг генератора составлял 1, 2 и 8 тактов. Дополнительный запуск с нижней границей 106 и верхней 95 завершился ожидаемой диагностикой.

7.2. Временные диаграммы

В запуске с шагом 8 тактов один профиль занимает 2,56 мкс. Сначала фиксируются веса 100, 80 и 100. После сброса последовательность начинается снова с годного изделия. Оба импульса соответствуют весу 80.

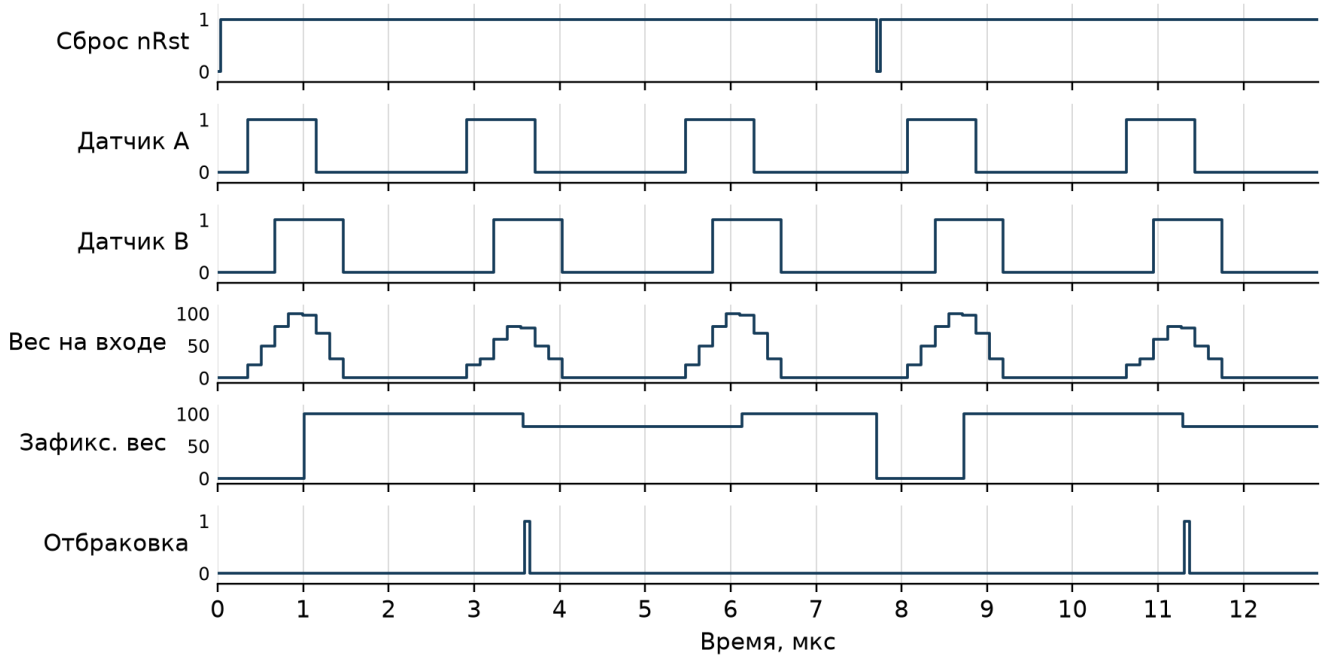


Рис. 4. Чередование изделий и повторный запуск после сброса

На позиции 6 вес снижается с 80 до 78. На следующем фронте ядро фиксирует предыдущий код 80, ещё через такт начинается отбраковка. На рисунке 5 импульс занимает три периода такта по 20 нс.

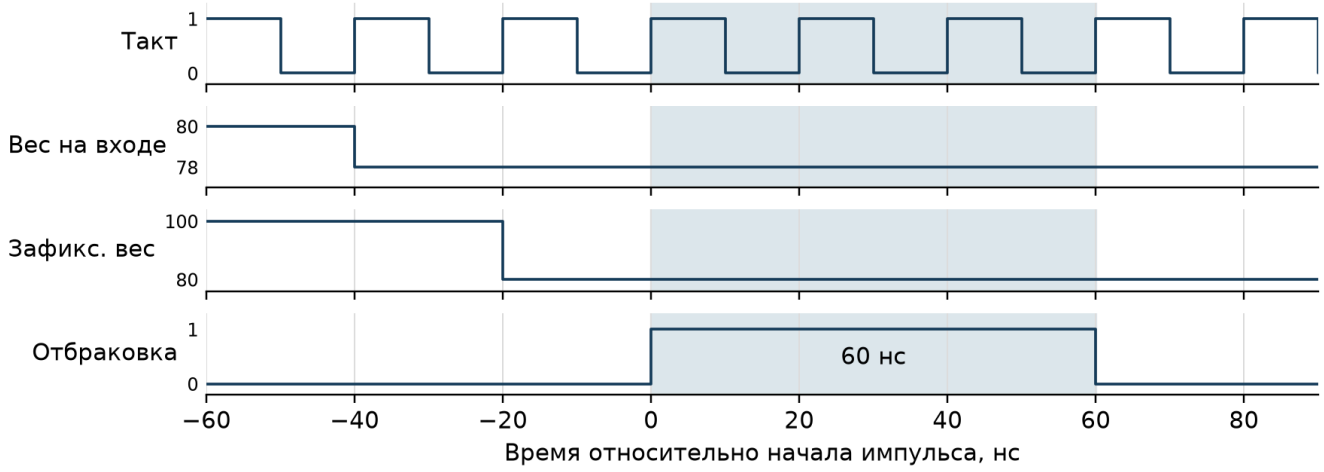


Рис. 5. Первый импульс отбраковки в увеличенном масштабе

8. Синтез и подготовка к проверке платы

Синтез GHDL 6.0.0 сформировал VHDL-нетлист демонстрационного верхнего уровня с параметрами платы. Проект `qmtech_demo.qpf` настроен на Cyclone 10 LP 10CL080YU484C8G. В QSF заданы контакты и стандарт 3.3-V LVTTL, в SDC - период 20 нс, неопределённость такта и исключения для асинхронной кнопки и светодиодов без внешнего такта приёмника.

Таблица 6. Назначения контактов QMTECH V03

Порт	Контакт ПЛИС	Цепь платы
clk50_i	G1	Генератор 50 МГц
nRst_btn_i	P4	Кнопка SW1, активный 0
conveyorLed_n_o	V10	Светодиод D1, активный 0
rejectLed_n_o	U9	Светодиод D2, активный 0

Источник назначений - принципиальная схема QMTECH Cyclone 10 Starter Kit V03.

Для этой редакции требуется полная компиляция в Quartus с проверкой отчёта TimeQuest на 50 МГц. Протоколы Quartus от 27.10.2026 относятся к предыдущей редакции. Физическое испытание требует платы QMTECH V03 и USB-Blaster. При проверке ожидается постоянно включённый D1, импульсы D2 по 100 мс раз в 3,2 с и повторный запуск профиля после SW1.

9. Заключение

Разработан пятисостоянийный автомат сортировки по весу и демонстрационный генератор двух профилей. Моделирование подтвердило границы 95–105, длительность импульса, обработку неполного прохода и восстановление чередования после сброса. Получен синтезированный нетлист. Размещение, временной анализ и проверка на физической плате для этой редакции остаются этапами аппаратной проверки.

10. Приложение

Полные VHDL-исходники, сценарий проверки и настройки проекта. Номера слева соответствуют исходным строкам; длинные строки перенесены при отображении.

Приложение 1. Файл conveyor_sorter.vhd

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_UNSIGNED.ALL;
4
5  entity conveyor_sorter is
6      generic (
7          WEIGHT_MIN          : natural range 0 to 255 := 95;
8          WEIGHT_MAX          : natural range 0 to 255 := 105;
9          REJECT_PULSE_CYCLES : positive := 5_000_000
10     );
11     port (
12         clk_i         : in  std_logic;
13         nRst_i        : in  std_logic;
14         sensorA_i     : in  std_logic;
15         sensorB_i     : in  std_logic;
16         weight_i      : in  std_logic_vector(7 downto 0);
17         conveyorRun_o : out std_logic;
18         reject_o      : out std_logic
19     );
20 end conveyor_sorter;
21
22 architecture rtl of conveyor_sorter is
23
24     type state_type is (WAIT_STATE, MEASURE_STATE, CHECK_STATE,
   REJECT_STATE, WAIT_CLEAR_STATE);
25
26     signal state_r          : state_type;
27     signal previousWeight_r : std_logic_vector(7 downto 0);
28     signal measuredWeight_r : std_logic_vector(7 downto 0);
29     signal rejectCount_r    : natural range 0 to REJECT_PULSE_CYCLES - 1;
30     signal centerFlag_r     : std_logic;
31
32     signal sensorsClear_s : boolean;
33     signal atCenter_s     : boolean;
34     signal captureWeight_s : boolean;
35     signal weightValid_s   : boolean;
36
37 begin
38
39     assert WEIGHT_MIN <= WEIGHT_MAX
40         report "WEIGHT_MIN must not exceed WEIGHT_MAX" severity failure;
41
42     sensorsClear_s <= sensorA_i = '0' and sensorB_i = '0';
43     atCenter_s <= sensorA_i = '1' and sensorB_i = '1';
44     captureWeight_s <= (weight_i < previousWeight_r and (centerFlag_r =
   '1' or atCenter_s))
45                        or (sensorsClear_s and centerFlag_r = '1')
46                        when state_r = MEASURE_STATE else false;
47     weightValid_s <= measuredWeight_r >= WEIGHT_MIN and measuredWeight_r
   <= WEIGHT_MAX
48                      when state_r = CHECK_STATE else false;
49
50     state_transitions : process(clk_i, nRst_i)
51     begin
52         if nRst_i = '0' then
53             state_r <= WAIT_STATE;
54         elsif rising_edge(clk_i) then
55             case state_r is
56                 when WAIT_STATE =>
57                     if sensorA_i = '1' or sensorB_i = '1' then
58                         state_r <= MEASURE_STATE;
59                     end if;
60                 when MEASURE_STATE =>
61                     if captureWeight_s then
62                         state_r <= CHECK_STATE;
63                     elsif sensorsClear_s then
64                         -- Изделие ушло, не достигнув центра.
65                         state_r <= WAIT_STATE;
```

```
                    end if;
                when CHECK_STATE =>
                    if weightValid_s then
                        state_r <= WAIT_CLEAR_STATE;
                    else
                        state_r <= REJECT_STATE;
                    end if;
                when REJECT_STATE =>
                    if rejectCount_r = 0 then
                        state_r <= WAIT_CLEAR_STATE;
                    end if;
                when WAIT_CLEAR_STATE =>
                    if sensorsClear_s then
                        state_r <= WAIT_STATE;
                    end if;
            end case;
        end if;
    end process;

    data_registers : process(clk_i, nRst_i)
    begin
        if nRst_i = '0' then
            previousWeight_r <= (others => '0');
            measuredWeight_r <= (others => '0');
            rejectCount_r    <= 0;
            centerFlag_r     <= '0';
        elsif rising_edge(clk_i) then
            if captureWeight_s then
                measuredWeight_r <= previousWeight_r;
            elsif state_r = WAIT_STATE or state_r = MEASURE_STATE then
                previousWeight_r <= weight_i;
            end if;

            -- Первый из N тактов отбраковки начинается при входе в
состояние.
            -- Загрузка N-1 обеспечивает верную длительность и при N=1.
            if state_r = CHECK_STATE and not weightValid_s then
                rejectCount_r <= REJECT_PULSE_CYCLES - 1;
            elsif state_r = REJECT_STATE and rejectCount_r /= 0 then
                rejectCount_r <= rejectCount_r - 1;
            end if;

            if state_r = WAIT_STATE then
                -- Оба датчика могут сработать уже на первом отсчёте
изделия.
                centerFlag_r <= sensorA_i and sensorB_i;
            elsif state_r = MEASURE_STATE and atCenter_s then
                centerFlag_r <= '1';
            end if;
        end if;
    end process;

    conveyorRun_o <= '1';
    reject_o <= '1' when state_r = REJECT_STATE else '0';

end rtl;
```

Приложение 2. Файл conveyor_sorter_top.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity conveyor_sorter_top is
    port (
        clk50_i       : in  std_logic;
        nRst_btn_i    : in  std_logic;
        sensorA_i     : in  std_logic;
        sensorB_i     : in  std_logic;
        weight_i      : in  std_logic_vector(7 downto 0);
        conveyorRun_o : out std_logic;
        reject_o      : out std_logic;
        conveyorLed_n_o : out std_logic;
        rejectLed_n_o   : out std_logic
    );
end conveyor_sorter_top;

architecture Structural of conveyor_sorter_top is

    signal resetSync_r      : std_logic_vector(1 downto 0);
    signal sensorASync_r    : std_logic_vector(1 downto 0);
```

```
    signal sensorBSync_r    : std_logic_vector(1 downto 0);

    signal weightMeta_r     : std_logic_vector(7 downto 0);
    signal weightSync_r     : std_logic_vector(7 downto 0);
    signal weightPrevious_r : std_logic_vector(7 downto 0);
    signal weightStable_r   : std_logic_vector(7 downto 0);

    signal nRst_s           : std_logic;
    signal conveyorRun_s    : std_logic;
    signal reject_s         : std_logic;

begin

    process(clk50_i, nRst_btn_i)
    begin

        if nRst_btn_i = '0' then

            resetSync_r <= (others => '0');

        elsif rising_edge(clk50_i) then

            resetSync_r(0) <= '1';
            resetSync_r(1) <= resetSync_r(0);

        end if;

    end process;

    nRst_s <= resetSync_r(1);

    process(clk50_i, nRst_s)
    begin

        if nRst_s = '0' then

            sensorASync_r    <= (others => '0');
            sensorBSync_r    <= (others => '0');
            weightMeta_r     <= (others => '0');
            weightSync_r     <= (others => '0');
            weightPrevious_r <= (others => '0');
            weightStable_r   <= (others => '0');

        elsif rising_edge(clk50_i) then

            sensorASync_r(0) <= sensorA_i;
            sensorASync_r(1) <= sensorASync_r(0);

            sensorBSync_r(0) <= sensorB_i;
            sensorBSync_r(1) <= sensorBSync_r(0);

            weightMeta_r <= weight_i;
            weightSync_r <= weightMeta_r;

            if weightSync_r = weightPrevious_r then
                weightStable_r <= weightSync_r;
            end if;

            weightPrevious_r <= weightSync_r;

        end if;

    end process;

    dut_inst : entity work.conveyor_sorter
        generic map (
            WEIGHT_MIN          => 95,
            WEIGHT_MAX          => 105,
            REJECT_PULSE_CYCLES => 5_000_000
        )
        port map (
            clk_i         => clk50_i,
            nRst_i        => nRst_s,
            sensorA_i     => sensorASync_r(1),
            sensorB_i     => sensorBSync_r(1),
            weight_i      => weightStable_r,
            conveyorRun_o => conveyorRun_s,
            reject_o      => reject_s
        );

```

```
    conveyorRun_o <= conveyorRun_s;
    reject_o      <= reject_s;

    conveyorLed_n_o <= not conveyorRun_s;
    rejectLed_n_o   <= not reject_s;

end Structural;
```

Приложение 3. Файл flow_tester.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity flow_tester is
    generic (STEP_CYCLES : positive := 5_000_000);
    port (
        clk_i     : in  std_logic;
        nRst_i    : in  std_logic;
        sensorA_o : out std_logic;
        sensorB_o : out std_logic;
        weight_o  : out std_logic_vector(7 downto 0)
    );
end flow_tester;

architecture rtl of flow_tester is
    constant LAST_POSITION : natural := 15;
    subtype position_type is natural range 0 to LAST_POSITION;
    subtype weight_type is std_logic_vector(7 downto 0);

    type sample_type is record
        sensorA      : std_logic;
        sensorB      : std_logic;
        validWeight  : weight_type;
        rejectWeight : weight_type;
    end record;
    type profile_type is array (position_type) of sample_type;

    -- Датчики A и B, затем вес годного и бракованного изделия.
    constant EMPTY_SAMPLE : sample_type := ('0', '0', x"00", x"00");
    constant PROFILE : profile_type := (
        2 => ('1', '0', x"14", x"14"), -- 20 / 20
        3 => ('1', '0', x"32", x"1E"), -- 50 / 30
        4 => ('1', '1', x"50", x"3C"), -- 80 / 60
        5 => ('1', '1', x"64", x"50"), -- 100 / 80
        6 => ('1', '1', x"62", x"4E"), -- 98 / 78
        7 => ('0', '1', x"46", x"32"), -- 70 / 50
        8 => ('0', '1', x"1E", x"1E"), -- 30 / 30
        others => EMPTY_SAMPLE
    );

    signal stepCount_r     : natural range 0 to STEP_CYCLES - 1;
    signal nextPosition_r  : position_type;
    signal rejectProduct_r : std_logic;
    signal stepEnable_s    : boolean;
begin
    stepEnable_s <= stepCount_r = 0;

    counter_registers : process(clk_i, nRst_i)
    begin
        if nRst_i = '0' then
            stepCount_r <= STEP_CYCLES - 1;
            -- Позиция 0 уже выставлена сбросом. Следующей будет позиция
1.
            nextPosition_r <= LAST_POSITION - 1;
            rejectProduct_r <= '0';
        elsif rising_edge(clk_i) then
            if stepEnable_s then
                stepCount_r <= STEP_CYCLES - 1;
                if nextPosition_r = 0 then
                    nextPosition_r <= LAST_POSITION;
                    rejectProduct_r <= not rejectProduct_r;
                else
                    nextPosition_r <= nextPosition_r - 1;
                end if;
            else
                stepCount_r <= stepCount_r - 1;
            end if;
        end if;
    end process;

```

```
    output_registers : process(clk_i, nRst_i)
        variable sample_v : sample_type;
    begin
        if nRst_i = '0' then
            sensorA_o <= PROFILE(0).sensorA;
            sensorB_o <= PROFILE(0).sensorB;
            weight_o <= PROFILE(0).validWeight;
        elsif rising_edge(clk_i) then
            if stepEnable_s then
                sample_v := PROFILE(LAST_POSITION - nextPosition_r);
                sensorA_o <= sample_v.sensorA;
                sensorB_o <= sample_v.sensorB;
                if rejectProduct_r = '0' then
                    weight_o <= sample_v.validWeight;
                else
                    weight_o <= sample_v.rejectWeight;
                end if;
            end if;
        end if;
    end process;
end rtl;
```

Приложение 4. Файл qmtech_demo_top.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity qmtech_demo_top is
    port (
        clk50_i         : in  std_logic;
        nRst_btn_i      : in  std_logic;
        conveyorLed_n_o : out std_logic;
        rejectLed_n_o   : out std_logic
    );
end qmtech_demo_top;

architecture structural of qmtech_demo_top is
    -- При 50 МГц шаг профиля и импульс отбраковки длятся по 100 мс.
    constant DEMO_STEP_CYCLES : positive := 5_000_000;
    constant DEMO_REJECT_CYCLES : positive := 5_000_000;

    signal sensorA_s, sensorB_s, conveyorRun_s, reject_s : std_logic;
    signal resetSync_r : std_logic_vector(1 downto 0);
    signal nRst_s : std_logic;
    signal weight_s : std_logic_vector(7 downto 0);
begin
    -- Общий сброс: включение асинхронное, снятие по такту.
    reset_synchronizer : process(clk50_i, nRst_btn_i)
    begin
        if nRst_btn_i = '0' then
            resetSync_r <= (others => '0');
        elsif rising_edge(clk50_i) then
            resetSync_r(0) <= '1';
            resetSync_r(1) <= resetSync_r(0);
        end if;
    end process;
    nRst_s <= resetSync_r(1);

    flow_tester_inst : entity work.flow_tester
        generic map (STEP_CYCLES => DEMO_STEP_CYCLES)
        port map (
            clk_i     => clk50_i,
            nRst_i    => nRst_s,
            sensorA_o => sensorA_s,
            sensorB_o => sensorB_s,
            weight_o  => weight_s
        );

    conveyor_sorter_inst : entity work.conveyor_sorter
        generic map (REJECT_PULSE_CYCLES => DEMO_REJECT_CYCLES)
        port map (
            clk_i         => clk50_i,
            nRst_i        => nRst_s,
            sensorA_i     => sensorA_s,
            sensorB_i     => sensorB_s,
            weight_i      => weight_s,
            conveyorRun_o => conveyorRun_s,
            reject_o      => reject_s
        );

```

```
    conveyorLed_n_o <= not conveyorRun_s;
    rejectLed_n_o <= not reject_s;
end structural;
```

Приложение 5. Файл struct_tb.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity struct_tb is
    generic (REJECT_CYCLES : positive := 3);
end struct_tb;

architecture simulation of struct_tb is
    constant CLOCK_PERIOD : time := 20 ns;
    signal done_s        : boolean := false;
    signal clk_s         : std_logic := '0';
    signal nRst_s        : std_logic := '0';
    signal sensorA_s     : std_logic := '0';
    signal sensorB_s     : std_logic := '0';
    signal weight_s      : std_logic_vector(7 downto 0) := (others => '0');
    signal conveyorRun_s : std_logic;
    signal reject_s      : std_logic;
begin
    dut_inst : entity work.conveyor_sorter
        generic map (REJECT_PULSE_CYCLES => REJECT_CYCLES)
        port map (clk_i => clk_s, nRst_i => nRst_s, sensorA_i => sensorA_s, sensorB_i =>
    sensorB_s,
                  weight_i => weight_s, conveyorRun_o => conveyorRun_s, reject_o => reject_s);

    clock_generator : process
    begin
        while not done_s loop
            clk_s <= '0'; wait for CLOCK_PERIOD / 2;
            clk_s <= '1'; wait for CLOCK_PERIOD / 2;
        end loop;
        wait;
    end process;

    watchdog : process
    begin
        wait until done_s for 100 us;
        assert done_s report "struct_tb timeout" severity failure;
        wait;
    end process;

    stimulus_and_check : process
        procedure tick is
        begin
            wait until rising_edge(clk_s);
            wait for 1 ns;
        end procedure;

        procedure reset_dut is
        begin
            nRst_s <= '0';
            wait for 1 ns;
            assert conveyorRun_s = '1' and reject_s = '0'
                report "Reset must keep the conveyor enabled and reject inactive" severity
    failure;
            tick;
            nRst_s <= '1';
            tick;
        end procedure;

        procedure check_reject_pulse is
        begin
            for cycle in 1 to REJECT_CYCLES loop
                assert reject_s = '1'
                    report "Reject pulse ended before the configured duration" severity failure;
                tick;
            end loop;
            assert reject_s = '0'
                report "Reject pulse exceeded the configured duration" severity failure;
```

```
67          end procedure;
68
69          procedure check_product(target_weight : positive;
    expected_reject : std_logic) is
70          begin
71              weight_s <=
    IEEE.STD_LOGIC_ARITH.CONV_STD_LOGIC_VECTOR(target_weight,
        weight_s'length);
72              sensorA_s <= '1';
73              sensorB_s <= '0';
74              tick;
75              sensorB_s <= '1';
76              tick;
77              weight_s <=
    IEEE.STD_LOGIC_ARITH.CONV_STD_LOGIC_VECTOR(target_weight - 1,
        weight_s'length);
78              tick;
79              assert reject_s = '0'
80                  report "Reject must remain inactive during the check
    state" severity failure;
81              tick;
82              assert reject_s = expected_reject
83                  report "Product was classified incorrectly" severity
    failure;
84
85              if expected_reject = '1' then
86                  check_reject_pulse;
87              else
88                  tick;
89                  assert reject_s = '0'
90                      report "Valid product entered the reject state"
    severity failure;
91              end if;
92
93              tick;
94              assert reject_s = '0'
95                  report "Reject must remain inactive while waiting for
    sensor release" severity
        failure;
96              sensorA_s <= '0';
97              sensorB_s <= '0';
98              tick;
99              tick;
100             assert conveyorRun_s = '1' and reject_s = '0'
101                 report "Sorter did not return to the waiting state"
    severity failure;
102         end procedure;
103     begin
104         reset_dut;
105         check_product(94, '1');
106         check_product(95, '0');
107         check_product(105, '0');
108         check_product(106, '1');
109         check_product(80, '1');
110         check_product(255, '1');
111         check_product(1, '1');
112
113         reset_dut;
114         weight_s <= x"00";
115         sensorA_s <= '1';
116         sensorB_s <= '1';
117         -- Совместное срабатывание длится один такт, сразу при входе.
118         tick;
119         sensorA_s <= '0';
120         sensorB_s <= '0';
121         tick;
122         assert reject_s = '0' report "Zero weight must pass through
    CHECK_STATE" severity
        failure;
123         tick;
124         check_reject_pulse;
125         tick;
126         tick;
127         check_product(100, '0');
128
129         reset_dut;
130         weight_s <= x"6E";
131         sensorA_s <= '1';
132         tick;
133         sensorA_s <= '0';
```

```
        weight_s <= x"64";
        tick;
        tick;
        assert reject_s = '0'
            report "A pass without center detection must not produce a reject pulse" severity
    failure;

        -- Сразу после неполного прохода идёт отдельное изделие весом 80.
        -- Оставшийся между изделиями отсчёт 100 не должен быть зафиксирован.
        weight_s <= x"50";
        sensorA_s <= '1';
        sensorB_s <= '1';
        tick;
        assert reject_s = '0' report "Reject asserted on entry" severity failure;
        sensorA_s <= '0';
        sensorB_s <= '0';
        tick;
        assert reject_s = '0' report "Reject asserted before classification" severity failure;
        tick;
        check_reject_pulse;

        reset_dut;
        weight_s <= x"50";
        sensorA_s <= '1';
        tick;
        sensorB_s <= '1';
        tick;
        weight_s <= x"4F";
        tick;
        tick;
        assert reject_s = '1'
            report "Out-of-range product must enter the reject state" severity failure;
        nRst_s <= '0';
        wait for 1 ns;
        assert conveyorRun_s = '1' and reject_s = '0'
            report "Asynchronous reset must stop rejection immediately" severity failure;
        tick;
        nRst_s <= '1';
        sensorA_s <= '0';
        sensorB_s <= '0';
        tick;
        assert reject_s = '0'
            report "Sorter must remain reset after an interrupted reject pulse" severity
    failure;

        report "All conveyor sorter checks passed" severity note;
        done_s <= true;
        wait;
    end process;
end simulation;
```

Приложение 6. Файл flow_tester_tb.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity flow_tester_tb is
    generic (STEP_CYCLES : positive := 8);
end flow_tester_tb;

architecture simulation of flow_tester_tb is
    constant CLOCK_PERIOD : time := 20 ns;
    constant REJECT_CYCLES : positive := 3;
    -- На позиции 6 начинается спад. Следующие два фронта фиксируют вес и проверяют его.
    constant FIRST_REJECT_CYCLE : positive := 6 * STEP_CYCLES + 2;
    signal done_s : boolean := false;
    signal clk_s : std_logic := '0';
    signal nRst_s : std_logic := '0';
    signal sensorA_s, sensorB_s, conveyorRun_s, reject_s : std_logic;
    signal weight_s : std_logic_vector(7 downto 0);
    signal rejectPulses_s : natural := 0;
```

```
20  begin
21      clock_generator : process
22      begin
23          while not done_s loop
24              clk_s <= '0'; wait for CLOCK_PERIOD / 2;
25              clk_s <= '1'; wait for CLOCK_PERIOD / 2;
26          end loop;
27          wait;
28      end process;
29
30      generator_inst : entity work.flow_tester
31          generic map (STEP_CYCLES => STEP_CYCLES)
32          port map (clk_i => clk_s, nRst_i => nRst_s,
33                    sensorA_o => sensorA_s, sensorB_o => sensorB_s, weight_o
    => weight_s);
34
35      sorter_inst : entity work.conveyor_sorter
36          generic map (REJECT_PULSE_CYCLES => REJECT_CYCLES)
37          port map (clk_i => clk_s, nRst_i => nRst_s,
38                    sensorA_i => sensorA_s, sensorB_i => sensorB_s, weight_i
    => weight_s,
39                    conveyorRun_o => conveyorRun_s, reject_o => reject_s);
40
41      monitor : process
42          variable previousReject_v : std_logic := '0';
43          variable pulseCycles_v : natural := 0;
44      begin
45          wait until rising_edge(clk_s);
46          wait for 1 ns;
47          assert conveyorRun_s = '1' report "Conveyor must remain enabled"
    severity failure;
48          if nRst_s = '0' then
49              rejectPulses_s <= 0;
50              previousReject_v := '0';
51              pulseCycles_v := 0;
52              assert reject_s = '0' report "Reject must be inactive during
    reset" severity
    failure;
53          else
54              if reject_s = '1' then
55                  if previousReject_v = '0' then
56                      rejectPulses_s <= rejectPulses_s + 1;
57                  end if;
58                  pulseCycles_v := pulseCycles_v + 1;
59              elsif previousReject_v = '1' then
60                  assert pulseCycles_v = REJECT_CYCLES
61                      report "Reject pulse has incorrect duration" severity
    failure;
62                  pulseCycles_v := 0;
63              end if;
64              previousReject_v := reject_s;
65          end if;
66      end process;
67
68      watchdog : process
69      begin
70          wait until done_s for 1 ms;
71          assert done_s report "flow_tester_tb timeout" severity failure;
72          wait;
73      end process;
74
75      stimulus : process
76          procedure tick is
77          begin
78              wait until rising_edge(clk_s);
79              wait for 2 ns;
80          end procedure;
81
82          procedure check_product(productIndex : natural) is
83              variable position_v : natural;
84              variable expectedWeight_v : natural;
85              constant IS_REJECTED : boolean := productIndex mod 2 = 1;
86              type profile_type is array (0 to 15) of natural;
87              constant VALID_PROFILE : profile_type := (0, 0, 20, 50, 80,
    100, 98, 70, 30, 0, 0,
        0, 0, 0, 0, 0);
88              constant INVALID_PROFILE : profile_type := (0, 0, 20, 30, 60,
    80, 78, 50, 30, 0, 0,
        0, 0, 0, 0, 0);
89          begin
```

```
            for cycle in 1 to 16 * STEP_CYCLES loop
                tick;
                position_v := (cycle / STEP_CYCLES) mod 16;
                if IS_REJECTED then
                    expectedWeight_v := INVALID_PROFILE(position_v);
                else
                    expectedWeight_v := VALID_PROFILE(position_v);
                end if;
                assert conv_integer(weight_s) = expectedWeight_v
                    report "Weight profile differs at a clock edge"
severity failure;
                if position_v >= 2 and position_v <= 6 then
                    assert sensorA_s = '1' report "Sensor A missing"
severity failure;
                else
                    assert sensorA_s = '0' report "Unexpected sensor A"
severity failure;
                end if;
                if position_v >= 4 and position_v <= 8 then
                    assert sensorB_s = '1' report "Sensor B missing"
severity failure;
                else
                    assert sensorB_s = '0' report "Unexpected sensor B"
severity failure;
                end if;
                if IS_REJECTED and cycle >= FIRST_REJECT_CYCLE
                   and cycle < FIRST_REJECT_CYCLE + REJECT_CYCLES then
                    assert reject_s = '1' report "Reject pulse missing at
the expected edge"
    severity failure;
                else
                    assert reject_s = '0' report "Reject must be inactive
outside the pulse
    window" severity failure;
                end if;
            end loop;
            assert rejectPulses_s = (productIndex + 1) / 2
                report "Only alternate, invalid products must be rejected"
severity failure;
            assert reject_s = '0' report "Reject must end before the next
product" severity
    failure;
        end procedure;
    begin
        tick;
        tick;
        nRst_s <= '1';
        -- Годное, брак, годное; следующим должен быть брак.
        for product in 0 to 2 loop
            check_product(product);
        end loop;
        -- После сброса первым снова идёт годное изделие.
        nRst_s <= '0';
        tick;
        tick;
        nRst_s <= '1';
        check_product(0);
        check_product(1);
        report "All alternating flow checks passed" severity note;
        done_s <= true;
        wait;
    end process;
end simulation;
```

Приложение 7. Файл test_alternating.do

```
onerror {quit -code 1 -force}
onbreak {quit -code 1 -force}
set source_dir [file dirname [file normalize [info script]]]
file mkdir [file join $source_dir simulation alternating]
cd [file join $source_dir simulation alternating]
if {![file exists work]} {vlib work}
vmap work work
foreach source {conveyor_sorter.vhd flow_tester.vhd
conveyor_sorter_top.vhd qmtech_demo_top.vhd
    struct_tb.vhd flow_tester_tb.vhd} {
    vcom -2008 [file join $source_dir $source]
}
set BreakOnAssertion 2
foreach cycles {1 2 3 8} {
```

```
    vsim -onfinish stop work.struct_tb -gREJECT_CYCLES=$cycles
    run -all
    quit -sim
}
foreach cycles {1 2 8} {
    vsim -onfinish stop work.flow_tester_tb -gSTEP_CYCLES=$cycles
    run -all
    quit -sim
}
```

Приложение 8. Файл qmtech_demo.qsf

```
set_global_assignment -name FAMILY "Cyclone 10 LP"
set_global_assignment -name DEVICE 10CL080YU484C8G
set_global_assignment -name TOP_LEVEL_ENTITY qmtech_demo_top
set_global_assignment -name VHDL_INPUT_VERSION VHDL_2008
set_global_assignment -name VHDL_FILE conveyor_sorter.vhd
set_global_assignment -name VHDL_FILE flow_tester.vhd
set_global_assignment -name VHDL_FILE qmtech_demo_top.vhd
set_global_assignment -name SDC_FILE qmtech_demo.sdc
set_location_assignment PIN_G1 -to clk50_i
set_location_assignment PIN_P4 -to nRst_btn_i
set_location_assignment PIN_V10 -to conveyorLed_n_o
set_location_assignment PIN_U9 -to rejectLed_n_o
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to clk50_i
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to nRst_btn_i
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to
conveyorLed_n_o
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to rejectLed_n_o

set_instance_assignment -name PARTITION_HIERARCHY root_partition -to | -
section_id Top
set_global_assignment -name PARTITION_NETLIST_TYPE_SOURCE -section_id Top
set_global_assignment -name PARTITION_FITTER_PRESERVATION_LEVEL
PLACEMENT_AND_ROUTING
    -section_id Top
set_global_assignment -name PARTITION_COLOR 16764057 -section_id Top
set_global_assignment -name LAST_QUARTUS_VERSION "25.1std.0 Standard
Edition"
set_global_assignment -name NUM_PARALLEL_PROCESSORS 4
# Светодиоды подключены через резисторы 1 кОм (схема QMTECH V03).
set_instance_assignment -name CURRENT_STRENGTH_NEW "4MA" -to
conveyorLed_n_o
set_instance_assignment -name CURRENT_STRENGTH_NEW "4MA" -to rejectLed_n_o
```

Приложение 9. Файл qmtech_demo.qpf

```
QUARTUS_VERSION = "22.1std"
PROJECT_REVISION = "qmtech_demo"
```

Приложение 10. Файл qmtech_demo.sdc

```
create_clock -name clk50 -period 20.000 [get_ports {clk50_i}]
derive_clock_uncertainty
# Кнопка асинхронно включает сброс.
# Снятие сброса синхронизируется через resetSync_r.
set_false_path -from [get_ports {nRst_btn_i}]
# Светодиоды не требуют привязки к внешнему такту.
set_false_path -to [get_ports {conveyorLed_n_o rejectLed_n_o}]
```